Advanced Databases

Assignment 2

Name	ID
Ziad Tarek	20236043
Haneen Soliman	20236032
Mohamed Ahmed	20237011
Shaza Moatasem	20236050
Rabab Mohamed	20237004

Introduction

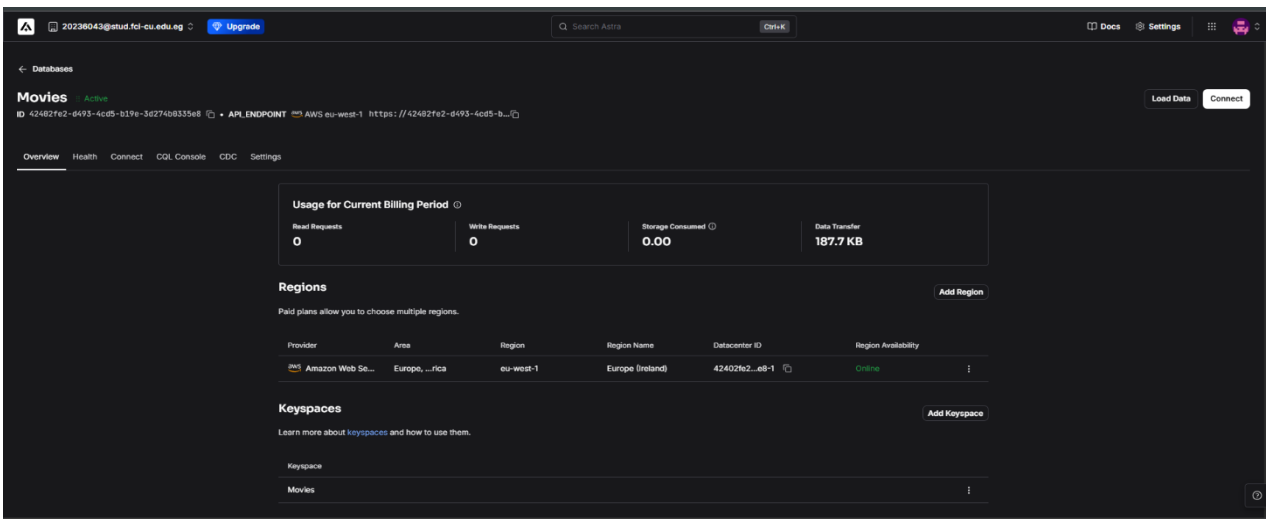
Apache Cassandra is a NoSQL distributed database that is designed to store large amounts of data while ensuring high availability and reliability. It follows a decentralized architecture, which makes it suitable for applications that require scalability and fault tolerance.

In this assignment, Cassandra is implemented using **DataStax Astra DB** to create and manage a database for movie information. The tasks include creating a keyspace and table, inserting and updating movie data, working with images stored as blobs, applying Time To Live (TTL) constraints, and querying data using both CQL and Python. The goal of this assignment is to gain practical experience with Cassandra data modeling and interacting with the database through application-level code.

Task 1: Database and Keyspace Creation

A database named **Movies** was created using **DataStax Astra DB**. Inside this database, a keyspace also named **Movies** was defined with a **Replication Factor of 3** using the **NetworkTopologyStrategy**. This configuration ensures that data is replicated across multiple nodes, providing high availability and fault tolerance.

The screenshot below shows the successfully created database and keyspace, along with the active status and the logged-in email address as required.

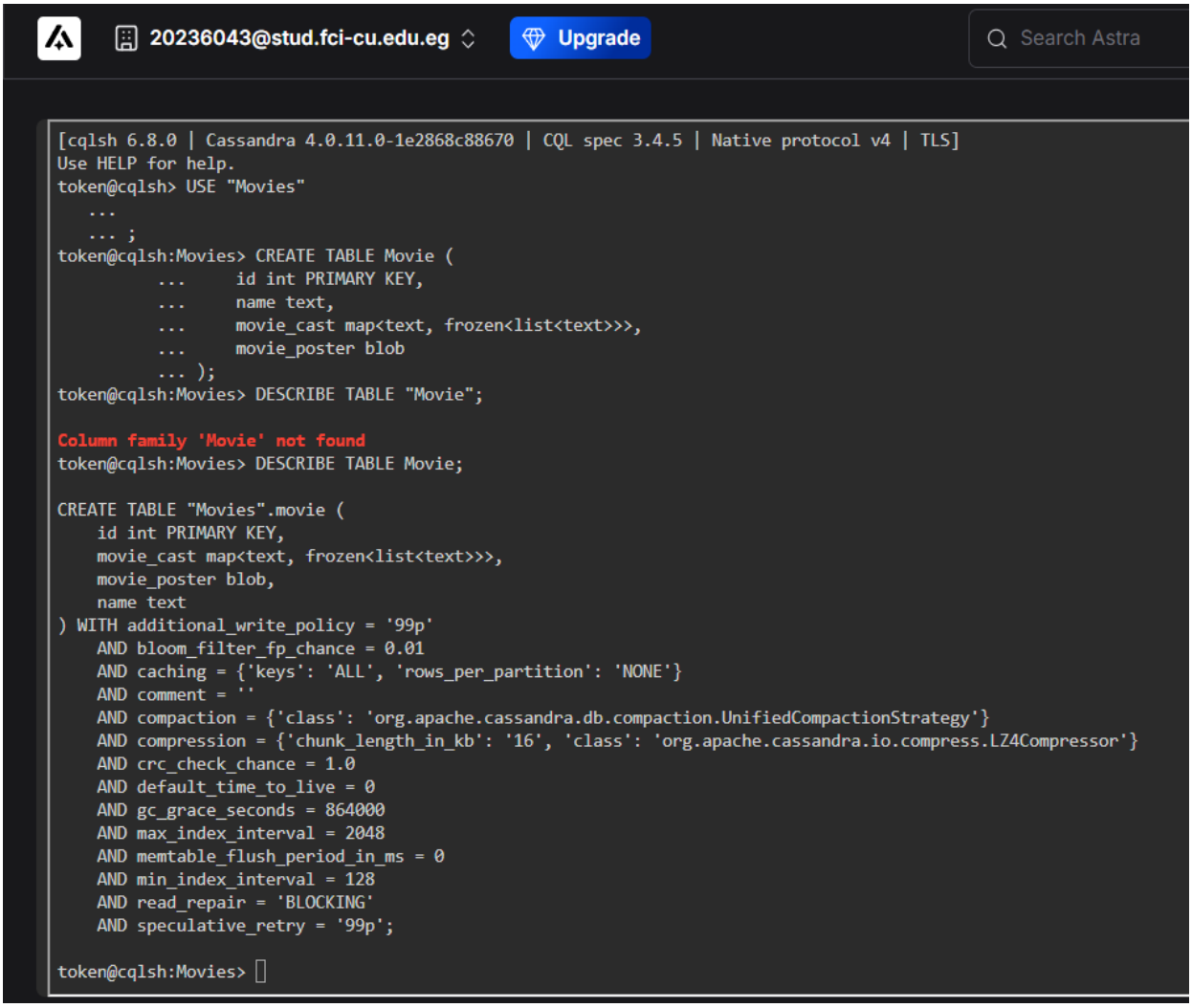



```
Connected as 20236043@stud.fci-cu.edu.eg.
Connected to cndb at cassandra.ingress:9042.
[cqlsh 6.8.0 | Cassandra 4.0.11.0-1e2868c88670 | CQL spec 3.4.5 | Native protocol v4 | TLS]
Use HELP for help.
token@cqlsh> DESCRIBE KEYSPACE "Movies"

CREATE KEYSPACE "Movies" WITH replication = {'class': 'NetworkTopologyStrategy', 'eu-west-1': '3'}  AND durable_writes = true;

CREATE TABLE "Movies".movie (
    id int PRIMARY KEY,
    movie_cast map<text, frozen<list<text>>>,
    movie_poster blob,
    name text
) WITH additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.UnifiedCompactionStrategy'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Task 2 & 3: Movie Table Creation and Schema Check

A column-family named **Movie** was created inside the **Movies** keyspace using CQL. The table includes the required columns: a primary key (id), movie name, a (movie_cast) map to store directors, actors, and music contributors, and a (movie_poster) column with blob data type for storing images.

The table was successfully created and verified using the DESCRIBE TABLE command.

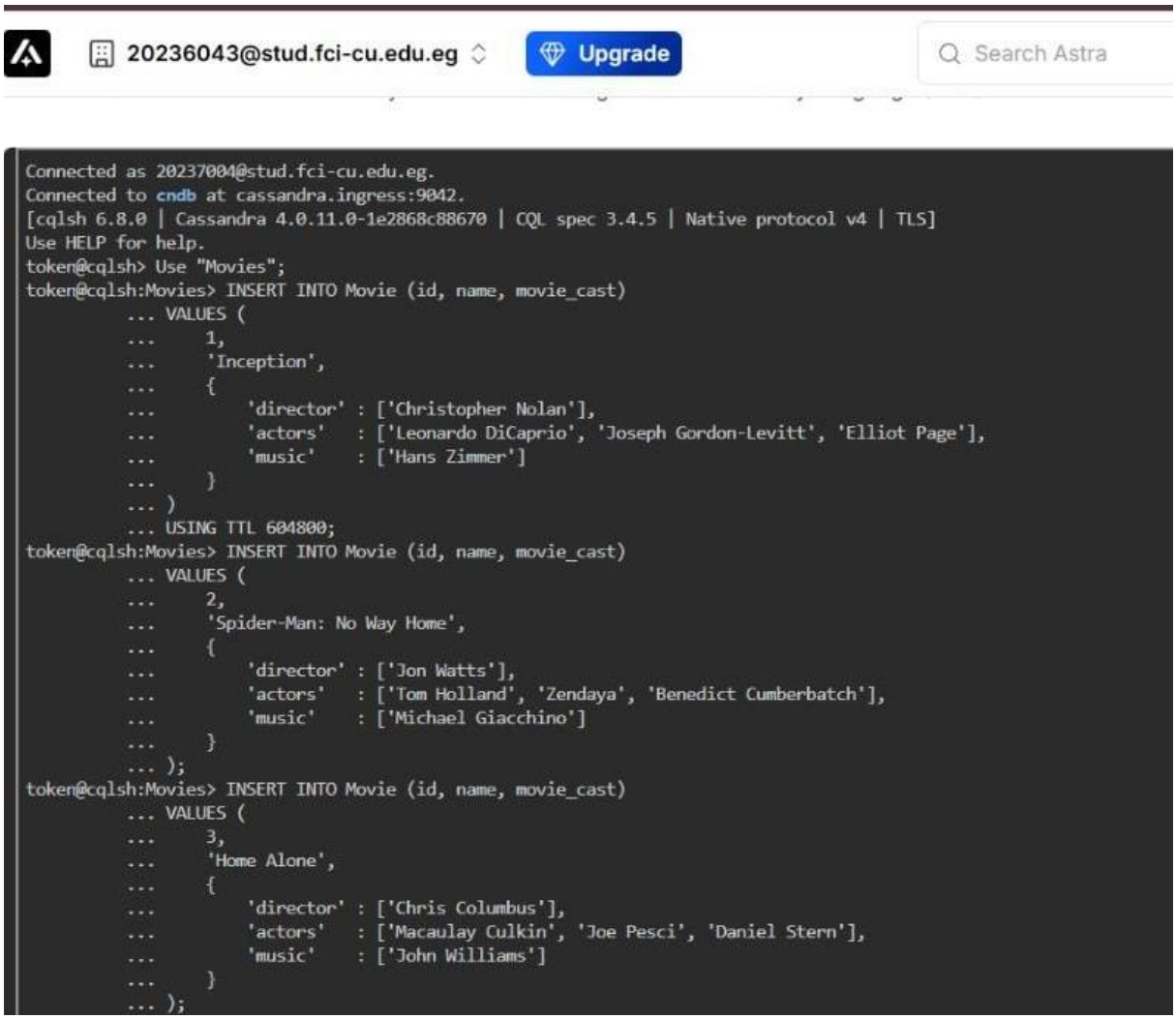



```
[cqlsh 6.8.0 | Cassandra 4.0.11.0-1e2868c88670 | CQL spec 3.4.5 | Native protocol v4 | TLS]
Use HELP for help.
token@cqlsh> USE "Movies"
   ...
   ... ;
token@cqlsh:Movies> CREATE TABLE Movie (
          ...     id int PRIMARY KEY,
          ...     name text,
          ...     movie_cast map<text, frozen<list<text>>>,
          ...     movie_poster blob
          ... );
token@cqlsh:Movies> DESCRIBE TABLE "Movie";

Column family 'Movie' not found
token@cqlsh:Movies> DESCRIBE TABLE Movie;

CREATE TABLE "Movies".movie (
    id int PRIMARY KEY,
    movie_cast map<text, frozen<list<text>>>,
    movie_poster blob,
    name text
) WITH additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.UnifiedCompactionStrategy'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';

token@cqlsh:Movies>
```

Task 4: Inserting Movie Data with TTL

Three real movies were inserted into the **Movie** table using CQL. Each record includes the movie ID, name, and cast information (director, actors, and music).

The first movie, **Inception**, was inserted with a **TTL of 7 days**, meaning the row will automatically expire after this period. The other two movies were inserted without a TTL. A SELECT query was used to confirm that all records were inserted correctly.



4.1: Displaying the Inserted Movies

After inserting the three movies into the **Movie** table, a SELECT * FROM Movie; query was executed to display all stored records. The output confirms that all three movies were successfully inserted, along with their cast information. At this stage, the movie_poster column contains null values, since the images are added later using Python.

```
          ... );
token@cqlsh:Movies> SELECT * FROM Movie;

 id | movie_cast                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | movie_poster | name
----+-----------------------
```

Task 5: Python Integration with Cassandra

5.1 Connecting to the KeySpace

A Python script was used to connect to the **Movies** keyspace in DataStax Astra DB. The connection was established using the secure connect bundle and client credentials provided by Astra DB. Successful connection to the database was confirmed through a console message.

```python
from cassandra.cluster import Cluster
from cassandra.auth import PlainTextAuthProvider
import json
cloud_config = {
    'secure_connect_bundle': 'secure-connect-movies.zip'
}
with open("20237011@stud.fci-cu.edu.eg-token.json", encoding="utf-8") as f:
    secrets = json.load(f)


CLIENT_ID = secrets["clientId"]
CLIENT_SECRET = secrets["secret"]

auth_provider = PlainTextAuthProvider(
    username=CLIENT_ID,
    password=CLIENT_SECRET
)

cluster = Cluster(
    cloud=cloud_config,
    auth_provider=auth_provider
)

session = cluster.connect("Movies")
print("Connected to Astra DB successfully")
```

✓ [5] 12s 224ms

```
Connected to Astra DB successfully
```

5.2 Encoding Movie Posters and Updating Blob Column

A Python function was implemented to read movie poster images from a local folder and convert them into binary format (blob). Another function then updates the movie_poster column in the **Movie** table for each movie using its ID. This process was applied to all three inserted movies.

```python
def encode_image_to_blob(image_path):
    with open(image_path, "rb") as file:
        return file.read()


def update_movie_poster(movie_id, image_path):
    image_blob = encode_image_to_blob(image_path)

    query = """
    UPDATE Movie
    SET movie_poster = %s
    WHERE id = %s
    """
    session.execute(query, (image_blob, movie_id))
    print(f"Poster updated for movie ID {movie_id}")


def show_movie_posters_status():
    query = "SELECT id, name, movie_poster FROM Movie"
    rows = session.execute(query)

    for row in rows:
        has_poster = row.movie_poster is not None
        print(f"ID={row.id}, Name={row.name}, Poster exists={has_poster}")
```

5.3 Decoding the Movie Poster

To verify that the images were stored correctly, a decoding function was implemented. This function retrieves the blob data from the database and writes it back as an image file in the output folder. The decoded images confirm successful storage and retrieval of blob data.

```python
def decode_movie_poster(movie_id, output_path):
    query = "SELECT movie_poster FROM Movie WHERE id = %s"
    row = session.execute(query, (movie_id,)).one()

    if row and row.movie_poster:
        with open(output_path, "wb") as img:
            img.write(row.movie_poster)
        print(f"Poster decoded: {output_path}")
    else:
        print("No poster found")


image_path1 = r"C:\Users\hp\python_project\Inception Poster 1.jpeg"
image_path2 = r"C:\Users\hp\python_project\Spiderman Poster 2.jpeg"
image_path3 = r"C:\Users\hp\python_project\Home Alone Poster 3.jpeg"

update_movie_poster(1, image_path1)
update_movie_poster(2, image_path2)
update_movie_poster(3, image_path3)
print()


show_movie_posters_status()


decode_movie_poster(1, "decoded_Inception.jpeg")
decode_movie_poster(2, "decoded_Spiderman.jpeg")
decode_movie_poster(3, "decoded_.Home_Alone.jpeg")
```

5.4 Code Execution and Results

The Python script was executed successfully. Console output shows that all movie posters were updated, verified as existing in the database, and decoded correctly into image files. This confirms proper integration between Python and Cassandra for handling blob data.

```
print("All posters processed successfully")

✓ [4] 5s 678ms

Poster updated for movie ID 1
Poster updated for movie ID 2
Poster updated for movie ID 3

ID=1, Cast={'actors': ['Leonardo DiCaprio', 'Joseph Gordon-Levitt', 'Elliot Page'], 'director': ['Christopher
Nolan'], 'music': ['Hans Zimmer']}, Poster exists=True,Name=Inception

ID=2, Cast={'actors': ['Tom Holland', 'Zendaya', 'Benedict Cumberbatch'], 'director': ['Jon Watts'], 'music':
['Michael Giacchino']}, Poster exists=True,Name=Spider-Man: No Way Home

ID=3, Cast={'actors': ['Macaulay Culkin', 'Joe Pesci', 'Daniel Stern'], 'director': ['Chris Columbus'], 'music':
['John Williams']}, Poster exists=True,Name=Home Alone

Poster decoded: decoded_Inception.jpeg
Poster decoded: decoded_Spiderman.jpeg
Poster decoded: decoded_.Home_Alone.jpeg
All posters processed successfully
```

Task 6: Querying Movies by Actor or Director and Query Limitations

6.1 Application-Level Search Using Python

A Python script was implemented to search for movies based on a user-provided **actor or director name**. Since Cassandra does not efficiently support text-based searching inside collection columns, the script retrieves all movie records and performs filtering at the application level.

```python
rows = session.execute(
    f"SELECT id, name, movie_cast, movie_poster FROM {TABLE}"
)

os.makedirs("output_posters", exist_ok=True)

found = False

for row in rows:
    cast_text = str(row.movie_cast).lower()
    if person_name in cast_text:
        found = True
        print("Movie:", row.name)

        safe_name = "".join(c for c in row.name if c.isalnum() or c in " _-")
        file_path = f"output_posters/{safe_name.replace(' ', '_')}.png"
        save_image(row.movie_poster, file_path)
```

Python virtual environment activated:

```
(venv) C:\Users\shaza\Desktop\Q6_Python>
```

Case 1:

This code handles “invalid input case”

```
if not found:
    print("No match found")
```

Result if no match found:

```
Enter actor or director name: Brad Pitt
No match found
```

Case 2:

When the user enters a valid actor or director name that exists in the database, the script scans the movie_cast field, identifies the matching movie, and displays the movie title. Once a match is found, the corresponding movie poster stored as a BLOB is decoded and saved locally as an image file. This confirms the correct functionality of the search logic for valid user input.

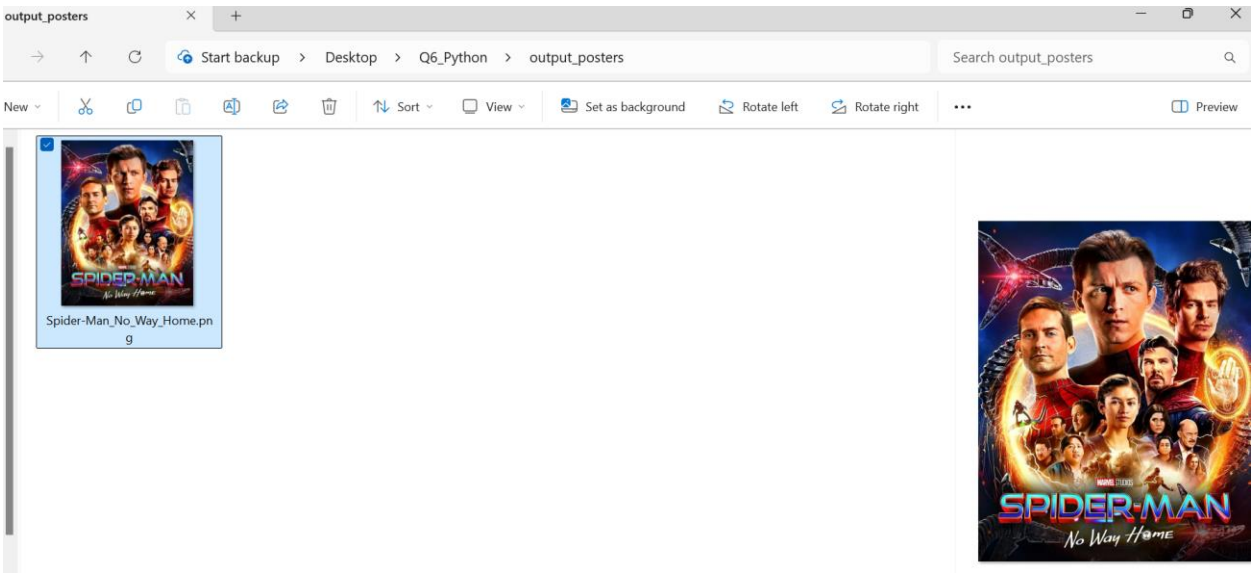
Movie 1: Spider-Man

Running Python search script with user input → Tom Holland

Result:

```
(venv) C:\Users\shaza\Desktop\Q6_Python>python query_movies.py
Enter actor or director name: Tom Holland
Movie: Spider-Man: No Way Home
Saved poster: output_posters/Spider-Man_No_Way_Home.png
```

Image saved:



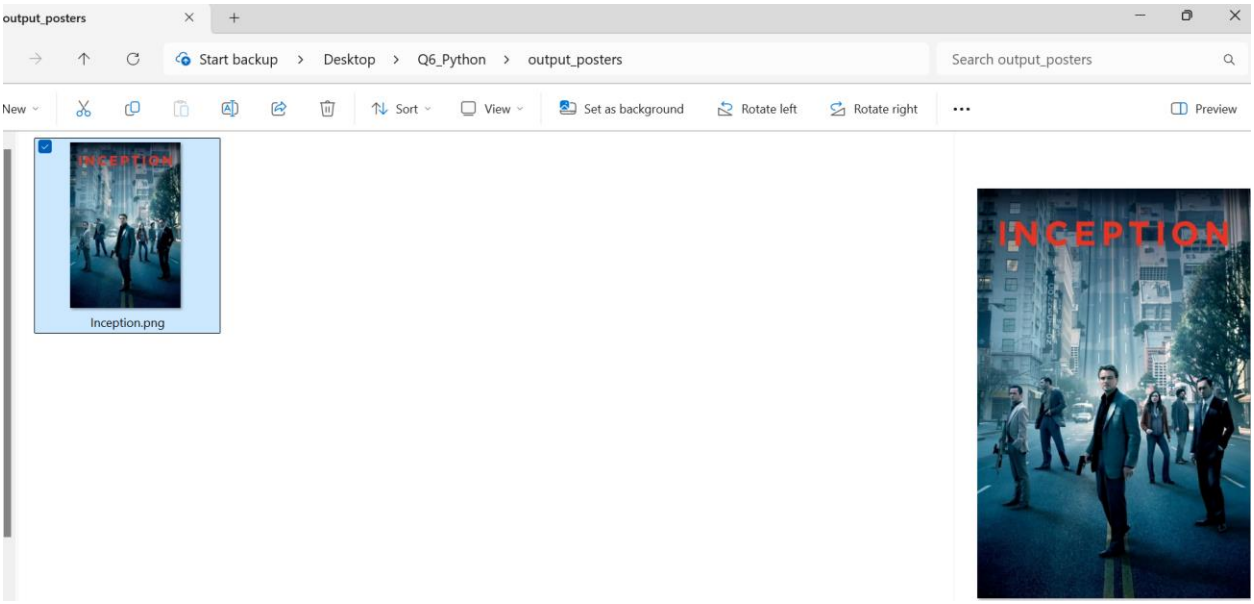
Movie 2: Inception

Running Python search script with user input → Leonardo DiCaprio

Result:

```
(venv) C:\Users\shaza\Desktop\Q6_Python>python query_movies.py
Enter actor or director name: Leonardo DiCaprio
Movie: Inception
Saved poster: output_posters/Inception.png
```

Image saved:



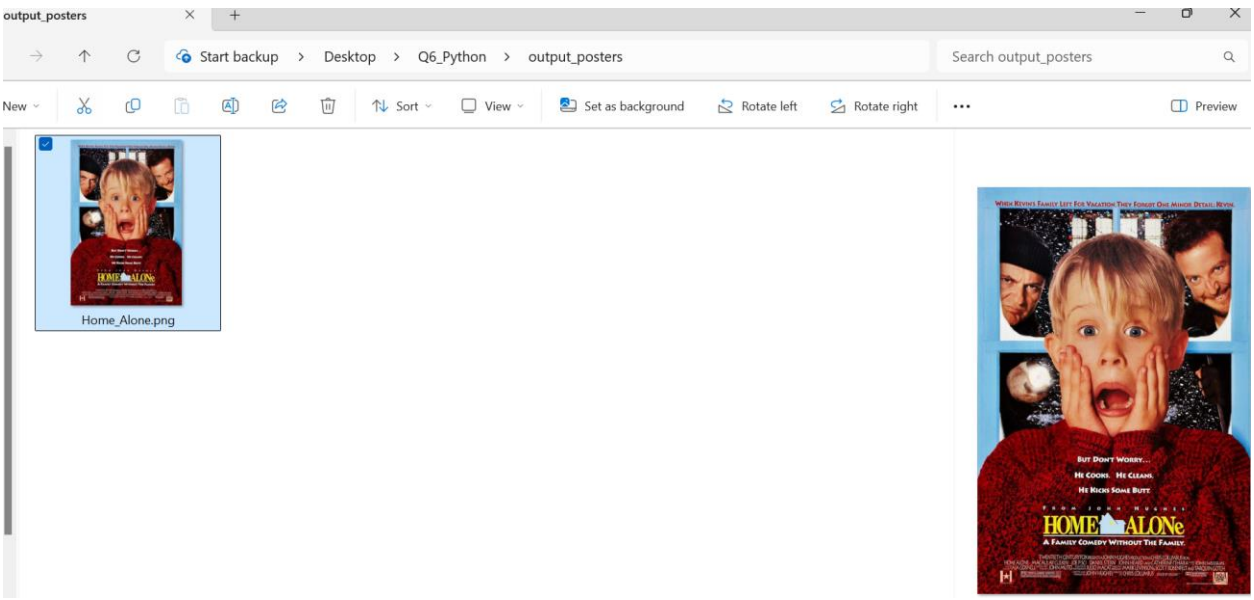
Movie 3: Home Alone

Running Python search script with user input → Macaulay Culkin

Result:

```
(venv) C:\Users\shaza\Desktop\Q6_Python>python query_movies.py
Enter actor or director name: Macaulay Culkin
Movie: Home Alone
Saved poster: output_posters/Home_Alone.png
```

Image saved:



In conclusion, the implemented Python-based search successfully retrieves movie records by actor or director name while overcoming Cassandra's limitations in text-based querying. This approach ensures accurate results and efficient handling of user input

Full Python Query Code:

```
import os
from cassandra.cluster import Cluster
from cassandra.auth import PlainTextAuthProvider
from PIL import Image
import io

# -------- CONFIG --------
KEYSPACE = "Movies"     # exact keyspace name
TABLE = "Movie"         # exact table name

ASTRA_CLIENT_ID = "AghzXyyJsspWbAqRDuPSsWWJ"
ASTRA_CLIENT_SECRET = "UT.KePjZqwRG6qJK1-+hW5GHuK7tr9QSlE6Nb3u-
BQaRZMjKB1,IiegCR.DTF+vly8a-3ir46BPpbarZA8PceZbFpUsHjJRyW-CKI+H6-glRtbcZhDzyw0ut-,-
_RELf"
SECURE_BUNDLE_PATH = "secure-connect-movies.zip"
# ------------------------

def connect_astra():
    auth = PlainTextAuthProvider(ASTRA_CLIENT_ID, ASTRA_CLIENT_SECRET)
    cluster = Cluster(
        cloud={"secure_connect_bundle": SECURE_BUNDLE_PATH},
        auth_provider=auth
    )
    session = cluster.connect()     # connect first
    session.set_keyspace("Movies")  # then set keyspace
    return session


def save_image(blob, filename):
    if blob is None:
        return
    image = Image.open(io.BytesIO(blob))
    image.save(filename)

def search_movie(person_name):
    session = connect_astra()
    person_name = person_name.lower()

    rows = session.execute(
        f"SELECT id, name, movie_cast, movie_poster FROM {TABLE}"
    )

    os.makedirs("output_posters", exist_ok=True)

    found = False
```

```
    for row in rows:
        cast_text = str(row.movie_cast).lower()
        if person_name in cast_text:
            found = True
            print("Movie:", row.name)

            safe_name = "".join(c for c in row.name if c.isalnum() or c in " _-")
            file_path = f"output_posters/{safe_name.replace(' ', '_')}.png"
            save_image(row.movie_poster, file_path)

            print("Saved poster:", file_path)

    if not found:
        print("No match found")


if __name__ == "__main__":
    name = input("Enter actor or director name: ")
    search_movie(name)
```

6.2 Cassandra Query Testing (LIKE vs CONTAINS)

LIKE Operator:
The LIKE operator is not supported in Apache Cassandra for searching within collection data types such as maps or lists. Since the movie_cast column is stored as a collection, LIKE cannot be used for partial text matching. This makes it unsuitable for searching actors or directors directly in CQL.

Testing LIKE Operator:

```
token@cqlsh> USE "Movies";
token@cqlsh:Movies> SELECT *
          ... FROM Movie
          ... WHERE movie_cast LIKE '%Nolan%';
InvalidRequest: Error from server: code=2200 [Invalid query] message="Collection column 'movie_c
ast' (map<text, frozen<list<text>>>) cannot be restricted by a 'LIKE' relation"
token@cqlsh:Movies>
```

CONTAINS Operator:
The CONTAINS operator can be used with collection columns in Cassandra, but it requires the use of ALLOW FILTERING, which may lead to inefficient queries, especially when dealing with large datasets. For this reason, CONTAINS is not recommended for implementing search functionality in production environments. As a result, application-level filtering using Python was adopted as a more reliable and efficient solution.

Testing CONTAINS Operator:

```
token@cqlsh:Movies> USE "Movies";
token@cqlsh:Movies> SELECT id, name, movie_cast
          ... FROM Movie
          ... WHERE movie_cast CONTAINS 'Christopher Nolan'
          ... ALLOW FILTERING;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Invalid STRING constant (C
hristopher Nolan) for "value(movie_cast)" of type frozen<list<text>>"
token@cqlsh:Movies>
```

Task 7: Updating Movie Actors List

In this task, the actors list of an existing movie was updated to include an additional actor. First, the current state of the **Movie** table was displayed to show the original cast information. Then, an UPDATE CQL statement was executed to modify the movie_cast map for the movie **Spider-Man: No Way Home** (ID = 2) by adding **Andrew Garfield** to the actors list.

After executing the update, the table was queried again to verify the change. The results confirm that the new actor was successfully added while all other movie data remained unchanged.

Before Update:

```
token@cqlsh> USE "Movies";
token@cqlsh:Movies> SELECT id, name, movie_cast FROM Movie;

 id | name                    | movie_cast
----+-------------------------+-------------------------
```

Task 8: Updating TTL and Observing Data Expiration

In this task, the **Time To Live (TTL)** feature in Cassandra was tested to observe how data expiration works at both the column level and the row level.

Approach 1: Column-Level TTL Using UPDATE

First, the table was queried to display the current state of all movies before applying any TTL. A **TTL of 3 seconds** was then applied to the first movie (**Inception**, ID = 1) using an UPDATE statement. After waiting for the specified duration, querying the table showed that the column values of the first movie became null, while the row itself still remained in the table. This demonstrates **column-level TTL behavior**, where only the updated columns expire and the row structure is preserved.

```
token@cqlsh:Movies> SELECT id, name, movie_cast FROM Movie;

 id | name                    | movie_cast
----+-------------------------+-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
  1 |                Inception |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
token@cqlsh:Movies> UPDATE Movie USING TTL 3
         ... SET name = 'Inception',
         ...     movie_cast = {'actors': ['Leonardo DiCaprio', 'Joseph Gordon-Levitt', 'Elliot Page'], 'director': ['Christopher Nolan'], 'music': ['Hans Zimmer']}
         ... WHERE id = 1;
token@cqlsh:Movies> SELECT id, name, movie_cast FROM Movie;

 id | name                    | movie_cast
----+-------------------------+-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
  1 |                    null |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                null
  2 | Spider-Man: No Way Home | {'actors': ['Tom Holland', 'Zendaya', 'Benedict Cumberbatch', 'Andrew Garfield'], 'director': ['Jon Watts'], 'music': ['Michael Giacchino']}
  3 |              Home Alone |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                {'actors': ['Macaulay Culkin', 'Joe Pesci', 'Daniel Stern'], 'director': ['Chris Columbus'], 'music': ['John Williams']}

(3 rows)
```

Approach 2: Row-Level TTL Using INSERT

To demonstrate row-level expiration, the first movie record was deleted to ensure a clean state. The movie was then reinserted using an INSERT statement with USING TTL 3. After waiting for 3 seconds, the table was queried again and the entire row for **Inception** was no longer present. This confirms **row-level TTL behavior**, where the whole row is removed once the TTL expires.

```
token@cqlsh:Movies> DELETE FROM Movie WHERE id = 1;
token@cqlsh:Movies> SELECT id, name FROM Movie;

 id | name
----+-------------------------
  2 | Spider-Man: No Way Home
  3 |               Home Alone

(2 rows)
```

```
token@cqlsh:Movies> INSERT INTO Movie (id, name, movie_cast)
          ... VALUES (1, 'Inception', {'actors': ['Leonardo DiCaprio', 'Joseph Gordon-Levitt', 'Elliot Page'], 'director': ['Christopher Nolan'], 'music': ['Hans Zimmer']})
          ... USING TTL 3;
token@cqlsh:Movies> SELECT id, name FROM Movie;

 id | name
----+-------------------------
  1 |               Inception
  2 | Spider-Man: No Way Home
  3 |               Home Alone

(3 rows)
token@cqlsh:Movies> SELECT id, name FROM Movie;

 id | name
----+-------------------------
  2 | Spider-Man: No Way Home
  3 |               Home Alone

(2 rows)
```

Conclusion

This assignment demonstrated the use of Apache Cassandra with DataStax Astra DB to manage a movie database. Key concepts such as data modeling, TTL behavior, blob handling, and Python integration were applied successfully. The work highlights how application-level logic can be used to overcome Cassandra’s querying limitations.